

## Practical 1: Multithreaded Counter using POSIX Threads

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
        counter++;

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter = %d\n", counter);

    return 0;
}
```

```
This message is shown once a day. To disable it please create the
/home/saireddy/.hushlogin file.
suresh@DESKTOP-QCI8C9M:~$ mkdir Thread_Practical
cd Thread_Practical
suresh@DESKTOP-QCI8C9M:~/Thread_Practical$ nano counter.c
suresh@DESKTOP-QCI8C9M:~/Thread_Practical$ gcc counter.c -o counter -pthread
./counter
Final Counter = 190808
suresh@DESKTOP-QCI8C9M:~/Thread_Practical$
```

## Practical 2: Counter using Mutex Locks

```
GNU nano 7.2
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for (int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Final Counter = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

```
suresh@DESKTOP-QCI8C9M:~/Thread_Practical$ nano mutex.c
suresh@DESKTOP-QCI8C9M:~/Thread_Practical$ gcc mutex.c -o mutex -pthread
./mutex
Final Counter = 200000
suresh@DESKTOP-QCI8C9M:~/Thread_Practical$ |
```

### Practical 3: Producer-Consumer using Semaphores

```
GNU nano 7.2
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t empty, full;
int buffer = 0;

void *producer(void *arg)
{
    for (int i = 1; i <= 5; i++)
    {
        sem_wait(&empty);

        buffer = i;
        printf("Produced: %d\n", buffer);
        sem_post(&full);
        sleep(1);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= 5; i++)
    {
        sem_wait(&full);

        printf("Consumed: %d\n", buffer);
        sem_post(&empty);
        sleep(1);
    }

    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, 1);
    sem_init(&full, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}
```

```

suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer$ nano producer.c
suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer$ gcc producer.c -o producer -pthread
suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer$ ./producer
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer$ |

```

## Practical 4: Deadlock using Threads

```

GNU nano 7.2
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t A, B;

void *thread1(void *arg)
{
    pthread_mutex_lock(&A);
    printf("Thread 1 locked A\n");

    sleep(1);

    pthread_mutex_lock(&B);
    printf("Thread 1 locked B\n");

    pthread_mutex_unlock(&B);
    pthread_mutex_unlock(&A);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&B);
    printf("Thread 2 locked B\n");

    sleep(1);

    pthread_mutex_lock(&A);
    printf("Thread 2 locked A\n");

    pthread_mutex_unlock(&A);
    pthread_mutex_unlock(&B);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&A, NULL);
    pthread_mutex_init(&B, NULL);

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}

```

```

cd Deadlock
suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer/Deadlock$ nano deadlock.c
suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer/Deadlock$ gcc deadlock.c -o deadlock -pthread
suresh@DESKTOP-QCI8C9M:~/Thread_Practical/ProducerConsumer/Deadlock$ ./deadlock
Thread 1 locked A
Thread 2 locked B
|

```